

```
        document.write("</pre>");  
    }
```

The pair of nested `for` loops iterate over every row and column on the board. The first `for` loop loops over the y-coordinates, and the second `for` loop loops over the x-coordinates. This is because the program needs to print all the columns of a single row before printing a newline character to move on to the next row.

The `if` statement checks whether the tile at the current x, y coordinates is the blank tile. If it is, the program prints two underscores with a trailing space. Otherwise, the code in the `else` block prints the tile number with a trailing space. The trailing space is what separates the tile numbers from one another on the screen. If the tile number is a single digit, the `rjust()` or `padStart()` method will insert an extra space so that the tile number is aligned with the two-digit numbers on the screen.

For example, say the scrambled puzzle on the left side of [Figure 12-1](#) is represented by this data structure:

```
[15, 2, 1, 12, 8, 5, 6, 11, 4, 9, 10, 7, 3, 14, 13, 0]
```

When the data structure is passed to `displayBoard()`, it prints the following text:

```
15  2  1 12  
 8  5  6 11  
 4  9 10  7  
 3 14 13 __
```

Creating a New Board Data Structure

Next, the `getNewBoard()` function returns a new board data structure with the tiles in their ordered, solved places. The Python code for the `getNewBoard()` function is as follows:

Python

```
def getNewBoard():  
    """Return a list that represents a new tile puzzle."""  
    board = []  
    for i in range(1, SIZE * SIZE):
```